Terraform CLI Cheat Sheet

Terraform Associate (004) exam reference

`terraform <subcommand> [options or flags]`

Help & Global Options

`terraform -help`	List all commands
`terraform --help`	Same as -help (also -h)
`terraform <subcommand> -help`	Help for a specific command
`terraform -chdir=envs/dev plan`	Run a command against another directory
`terraform version`	Show CLI version and provider versions in use
`terraform -install-autocomplete`	Enable tab-completion for commands (bash/zsh, restart shell after)
`terraform -uninstall-autocomplete`	Remove tab-completion

Format

`terraform fmt`	Format config files to the canonical style
`terraform fmt -recursive`	Include subdirectories
`terraform fmt -check`	Exit non-zero if files are unformatted (CI)
`terraform fmt -diff`	Show formatting changes

Init

`terraform init`	Download providers & modules, configure backend, create/update .terraform.lock.hcl
`terraform init -upgrade`	Upgrade providers/modules within version constraints
`terraform init -reconfigure`	Reinitialize backend, ignoring existing config
`terraform init -migrate-state`	Migrate state to a new backend
`terraform init -backend-config=backend.hcl`	Supply partial backend configuration
`terraform init -backend=false`	Skip backend initialization

Validate

`terraform validate`	Check config for syntax and internal consistency (needs init first)
`terraform validate -json`	Machine-readable output

Plan

`terraform plan`	Preview changes (dry run)
`terraform plan -out=myplan.tfplan`	Save the plan to a file for review/approval
`terraform plan -destroy`	Preview what destroy would do
`terraform plan -refresh-only`	Only detect drift between state and real infra
`terraform plan -replace=<addr>`	Plan to force recreate a resource
`terraform plan -target=<addr>`	Plan only a specific resource
`terraform plan -var="location=eastus"`	Set a variable from the CLI
`terraform plan -var-file=dev.tfvars`	Load variables from a file
`terraform plan -detailed-exitcode`	Exit codes: 0 = no changes, 1 = error, 2 = changes present

Apply

`terraform apply`	Standard apply: runs a plan, then prompts for confirmation
`terraform apply myplan.tfplan`	Apply a saved plan exactly as reviewed (no prompt)
`terraform apply -auto-approve`	Skip the confirmation prompt (use carefully)
`terraform apply -input=false`	Disable interactive input prompts (automation/CI/CD)
`terraform apply -parallelism=5`	Change the number of parallel operations (default: 10)
`terraform apply -refresh-only`	Update state to match real infra (replaces deprecated `terraform refresh`)
`terraform apply -replace=<addr>`	Force recreate a resource (replaces deprecated `terraform taint`)
`terraform apply -destroy`	Same as terraform destroy

Destroy

`terraform destroy`	Destroy all resources managed by this configuration
`terraform destroy -target=<resource_type>.<name>`	Destroy a single specific resource

Inspect State & Outputs

`terraform show`	Human-readable view of current state
`terraform show myplan.tfplan`	Inspect a saved plan
`terraform show -json`	Machine-readable output
`terraform output`	Show all outputs
`terraform output <name>`	Show a single output (reveals sensitive values)
`terraform output -raw <name>`	Plain string, no quotes (scripts)
`terraform output -json`	All outputs as JSON
`terraform console`	Interactive shell to test expressions/functions
`terraform graph | dot -Tsvg > graph.svg`	Export dependency graph (DOT) and render with Graphviz

State Management

`terraform state list`	List resources in state
`terraform state show <addr>`	Show details of one resource
`terraform state mv <src> <dest>`	Rename/move a resource in state (no recreate)
`terraform state rm <addr>`	Stop managing a resource (real resource NOT destroyed)
`terraform state pull`	Download and print remote state
`terraform state push`	Upload local state to remote (use with care)
`terraform state replace-provider <old> <new>`	Replace provider in state
`terraform import <addr> <id>`	Import existing resource into state (write the block yourself)
`terraform plan -generate-config-out=gen.tf`	With `import` blocks (v1.5+): auto-generate config
`terraform force-unlock <LOCK_ID>`	Remove a stuck state lock

> *No `terraform move` command - use `terraform state mv` or a `moved {}` block in config.*

Modules & Providers

`terraform get`	Download modules referenced in config
`terraform get -update`	Update modules
`terraform modules`	List declared modules (v1.10+)
`terraform providers`	Show providers required by config & modules
`terraform providers lock`	Update lock file (e.g. for multiple platforms)
`terraform providers mirror <dir>`	Copy providers locally for offline/air-gapped use
`terraform providers schema -json`	Print provider schemas

Workspaces

`terraform workspace list`	List workspaces (* = current)
`terraform workspace show`	Show current workspace
`terraform workspace new <name>`	Create and switch to a new workspace
`terraform workspace select <name>`	Switch workspace
`terraform workspace delete <name>`	Delete a workspace (cannot delete `default`)

HCP Terraform

`terraform login`	Get & save API token for app.terraform.io
`terraform login <hostname>`	Login to a Terraform Enterprise host
`terraform logout`	Remove saved credentials

Testing

`terraform test`	Run *.tftest.hcl test files (v1.6+)

Few Advanced Commands

Deprecated commands (know what replaces them)

`terraform taint <addr>`	DEPRECATED - mark resource for recreation → use: terraform apply -replace=<addr>
`terraform untaint <addr>`	DEPRECATED - remove taint mark
`terraform refresh`	DEPRECATED - update state from real infra → use: terraform apply -refresh-only

State locking

`terraform plan -lock=false`	Don't acquire a state lock (dangerous with shared state)
`terraform apply -lock-timeout=60s`	Wait up to 60s for the lock instead of failing immediately

Refresh behavior

`terraform plan -refresh=false`	Skip checking real infra (faster, but may use stale state)

Targeting & variables on apply/destroy

`terraform apply -target=<addr>`	Apply only a specific resource (exceptional fixes only)
`terraform apply -var="location=eastus"`	-var / -var-file also work on apply and destroy
`terraform destroy -var-file=dev.tfvars`	

Providers & HCP Terraform

`terraform providers lock -platform=linux_amd64 -platform=windows_amd64`	Record provider hashes for multiple OSes
`terraform init`	With a `cloud {}` block: connects to HCP Terraform workspaces

Metadata

`terraform metadata functions -json`	List all available built-in functions (JSON)

Config blocks instead of state commands (declarative, show up in the plan for review):

```
# Import existing resource (v1.5+) - instead of `terraform import`
import {
  to = azurerm_resource_group.main
  id = "/subscriptions/<sub-id>/resourceGroups/rg-demo"
}

# Rename/move resource (v1.1+) - instead of `terraform state mv`
moved {
  from = azurerm_virtual_network.old
  to   = azurerm_virtual_network.main
}

# Stop managing without destroying (v1.7+) - instead of `terraform state rm`
removed {
  from = azurerm_storage_account.legacy
  lifecycle {
    destroy = false
  }
}

# Connect to HCP Terraform
terraform {
  cloud {
    organization = "my-org"
    workspaces {
      name = "dev"
    }
  }
}
```

Environment Variables

Bash

`export TF_VAR_location="eastus"`	Set input variable `location`
`export TF_LOG=DEBUG`	Enable logging (TRACE|DEBUG|INFO|WARN|ERROR)
`export TF_LOG_PATH=./terraform.log`	Write logs to a file (needs TF_LOG)
`export TF_INPUT=0`	Disable interactive prompts
`export TF_CLI_ARGS_plan="-parallelism=5"`	Extra flags for a specific command
`export TF_WORKSPACE=dev`	Select workspace
`TF_LOG=DEBUG terraform plan`	Set only for a single command
`unset TF_LOG`	Turn logging off
`alias tf=terraform`	Shortcut: `tf init` instead of `terraform init` (add to ~/.bashrc to persist)

PowerShell

`$env:TF_VAR_location = "eastus"`	Set input variable `location`
`$env:TF_LOG = "DEBUG"`	Enable logging
`Remove-Item Env:TF_LOG`	Turn logging off
`Set-Alias tf terraform`	Shortcut: `tf init` instead of `terraform init` (add to $PROFILE to persist)